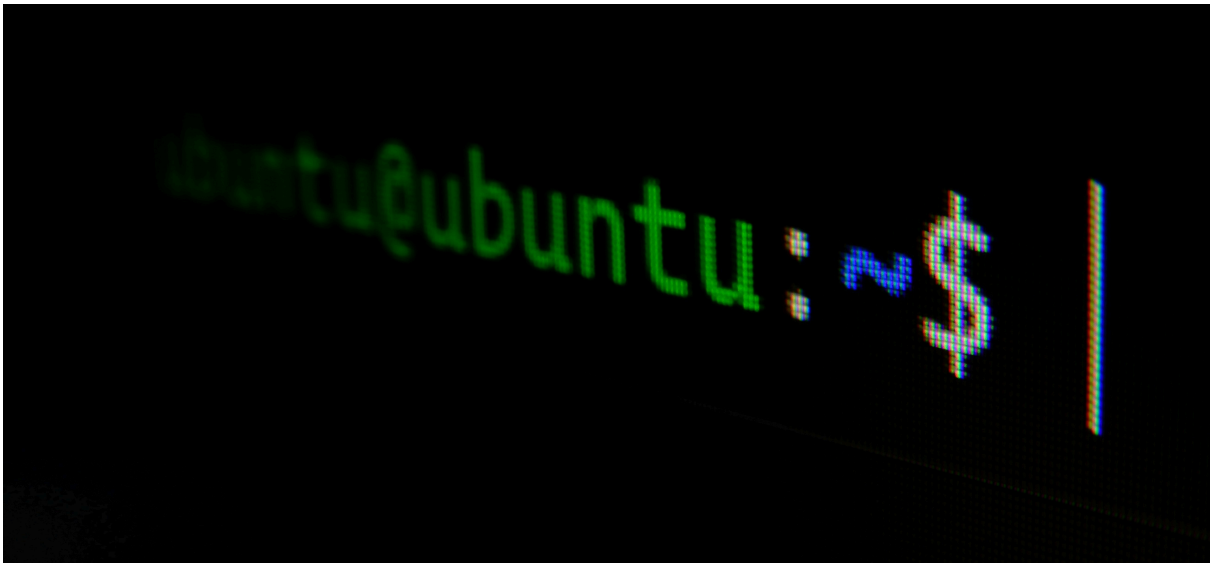

CHAPTER 15 - TASK AUTOMATION AND JOB SCHEDULING

Linux users frequently need to execute jobs, scripts, or various tasks on a recurring basis. Common examples include setting up automated system backups or implementing log file rotation procedures as discussed in earlier chapters. Security professionals might want to schedule vulnerability scanning scripts to run during off-hours when they're away from their workstation or attending other responsibilities. These scenarios demonstrate the value of automated job scheduling.

Automated task scheduling eliminates the need for manual intervention and allows you to utilize system resources during periods of low activity. System administrators and security practitioners often require certain applications or services to launch automatically during system startup. For instance, when working with penetration testing frameworks like Metasploit in conjunction with PostgreSQL databases, rather than manually initializing the database each time before launching Metasploit, you can configure PostgreSQL—or any service or application—to start automatically during the boot process.

This chapter will explore how to leverage the **cron daemon** and **crontab** utility to configure scripts for unattended execution. Additionally, you'll learn to implement startup scripts that execute automatically

during system initialization, ensuring that essential services are available throughout your security testing activities.

▼ [15.1] SCHEDULING AN AUTOMATED EVENT OR JOB

The **cron daemon** (**crond**) and **cron table** (**crontab**) serve as the primary mechanisms for automating recurring system tasks. The cron daemon operates continuously in the background, monitoring the cron table to determine which commands should execute at designated times. Through modifications to the cron table, you can configure tasks to run on specific schedules—whether daily at particular times, on certain dates, or at regular intervals spanning weeks or months.

Task scheduling is accomplished by adding entries to the cron table file, which resides at `/etc/crontab`. Each cron table entry consists of seven distinct fields: the initial five fields define the execution schedule, the sixth field identifies the user account under which the task will run, and the seventh field contains the complete file path to the command or script being executed.

The five scheduling fields correspond to specific time components in sequential order: minutes, hours, day of month, month, and day of week. All time values must be expressed numerically—for example, March is represented as "3" rather than spelled out. The day of week numbering system starts with 0 for Sunday and cycles through to 7, which also represents Sunday, creating a wrap around at both ends of the weekly cycle.

Time Representations for Use in the crontab

Field Time Unit	Representation
Minute	0 - 59
Hour	0 - 23
Day of the Month	1 - 31
Month	1 - 12
Day of the week	0 - 7

For instance, if you've developed a script to perform global vulnerability scanning for exposed ports and want it to execute automatically at 2:30 AM

each weeknight (Monday through Friday), you can configure this schedule using the crontab system. We'll examine the step-by-step process for adding this information to the crontab shortly, but let's first review the required formatting structure:

```
M  H  DOM  MON  DOW  USER  COMMAND
30 2  *    *    15   root  /root/myscanningscript
```

The **crontab** file provides clear column headers for reference. In our example, the first field specifies minutes (30), the second indicates hours (2), the fifth field designates weekdays (1-5 for Monday through Friday), the sixth field identifies the user account (root), and the seventh field contains the script's file path. The third and fourth fields use asterisks (*) because we want the script to execute every weekday regardless of the specific date or month. The fifth field demonstrates range notation using a hyphen (-) to specify consecutive days. For non-consecutive days, you would use commas—for example, Tuesday and Thursday would be written as "2,4."

Editing Crontab

To modify crontab, use the command "crontab -e" (edit option). The first time you run this command, it prompts you to select a text editor, with /bin/nano as the default option. Alternatively, you can open the crontab file directly in your preferred text editor using a command like: `nano /etc/crontab`

The crontab file contains system environment variables and existing scheduled tasks. To add a new recurring task, simply insert a new line following the established format and save the file.

Practical Example: Backup Scheduling

From a system administration perspective, automated backups are commonly scheduled during off-peak hours when system resources are abundant and user activity is minimal. Weekend overnight hours are ideal for this purpose. Rather than manually logging in at inconvenient times like 2 AM on Saturday night, you can configure automatic execution.

Important formatting notes: the hour field uses 24-hour time format (1 PM = 13:00), and days of the week range from Sunday (0) to Saturday (6). The asterisk (*) wildcard represents "any" or "all" values for that time component.

For example, to schedule a system backup script named "systembackup.sh" (stored in /bin directory) to run every Sunday at 2 AM using the "backup" user account, you would add this line to crontab:

```
00 2 * * 0 backup /bin/systembackup.sh
```

This entry translates to: execute at minute 00, hour 2, any day of month, any month, on Sunday (0), as the backup user, running the specified script.

If you wanted the backup to occur only on the 15th and 30th of each month regardless of the day of the week, you would modify the crontab entry accordingly:

```
00 2 15,30 * * backup /root/systembackup.sh
```

Notice that the day of month (DOM) field now contains "15,30". This instructs the system to execute the script exclusively on the 15th and 30th of each month, creating approximately bi-weekly intervals. When specifying multiple values for days, hours, or months, you must list them with comma separation, as demonstrated in this example.
Now, suppose your organization has stricter backup requirements and cannot tolerate losing even a single day's worth of data due to power failures or system crashes. In this scenario, you would need to implement nightly backups on weekdays by including the following entry:

```
 00 23 * * 15 backup /root/systembackup.sh
```

This scheduled task would execute at 11 PM (using 24-hour format as hour 23), on every date of the month, during every month, but restricted to Monday through Friday (days 1-5). Note that the weekday specification uses interval notation with a dash (-) to indicate the range from 1 to 5. Alternatively, this could be written as individual comma-separated values (1,2,3,4,5)—both formats function identically.

Scheduling the MySQL Scanner with Crontab

Now that you've grasped the fundamentals of job scheduling using crontab, let's apply this knowledge to automate the MySQLscanner.sh script from Chapter 8. This scanner identifies systems running MySQL by detecting open port 3306 connections.

To add your MySQLscanner.sh to the crontab configuration, you'll need to edit the crontab file and specify the job parameters, similar to the backup scheduling examples. We'll configure it to run during daytime hours while you're at work, ensuring it doesn't consume system resources when you're actively using your home computer. Add the following entry to your crontab:

```
00 9 * * * user /usr/share/MySQLsscanner.sh
```

We've configured the task to execute at minute 00, during the 9th hour (9 AM), on any day of the month (*), *during any month* (*), on any day of the week (*), running under a standard user account. Simply saving this crontab file will activate the scheduled job.

Now, suppose you want to take a more cautious approach and only execute this scanner during weekend hours at 2 AM when network monitoring is less likely to detect the activity. Additionally, you want to restrict its operation to summer months only (June through August). Your crontab entry would then be modified as follows:

```
00 2 * 68 0,6 user /usr/share/MySQLsscanner.sh
```

▼ BONUS [`crontab` shortcuts]:

Crontab Convenience Shortcuts and Simplified Scheduling

The crontab system includes several predefined shortcuts that eliminate the need to manually specify individual time, day, and month parameters for common scheduling patterns. These built-in macros provide a more intuitive and readable alternative to the traditional five-field time specification format. The available shortcuts encompass a comprehensive range of typical scheduling scenarios:

@yearly - Executes the task once per year, equivalent to "0 0 1 1 *" (January 1st at midnight)

@annually - Functions identically to @yearly, providing an alternative syntax for annual execution

@monthly - Runs the task once per month, equivalent to "0 0 1 * *" (first day of each month at midnight)

@weekly - Schedules weekly execution, equivalent to "0 0 * * 0" (every Sunday at midnight)

@daily - Performs daily execution, equivalent to "0 0 * * *" (every day at midnight)

@midnight - Functions identically to @daily, specifically emphasizing the midnight execution time

@noon - Executes the task daily at 12:00 PM, equivalent to "0 12 * * *"

@reboot - Triggers execution immediately after system startup, regardless of time or date

These shortcuts significantly simplify crontab entries and reduce the potential for syntax errors when configuring routine scheduling patterns. They make the crontab file more readable and maintainable, especially for users who may not be familiar with the traditional five-field time format.

Practical Implementation Example

To demonstrate the practical application of these shortcuts, consider scheduling the MySQL scanner to run automatically every night at midnight. Instead of writing the traditional format `0 0 * * * user /usr/share/MySQLscanner.sh`, you can use the more intuitive shortcut syntax:

```
@midnight user /usr/share/MySQLscanner.sh
```

This simplified format immediately conveys the scheduling intent without requiring knowledge of the underlying time field specifications, making system administration tasks more accessible and less error-prone.

▼ [15.2] USING RC SCRIPTS

System Initialization and Startup Scripts

During Linux system startup, a collection of initialization scripts automatically executes to establish the proper operating environment. These scripts are collectively referred to as rc (run command) scripts. The boot sequence begins when the kernel completes its initialization phase and finishes loading all necessary modules. At this point, the kernel launches a critical background process called init or init.d daemon. This daemon

subsequently executes a series of configuration scripts located in the /etc/init.d/rc directory. These startup scripts contain the commands required to initialize essential services that enable your Linux system to function as expected.

Understanding Linux Runlevels

Linux operating systems utilize a runlevel system that determines which services and processes should be activated during system startup. Each runlevel represents a different operational state with varying levels of functionality:

- **Runlevel 0**: Complete system shutdown/halt
- **Runlevel 1**: Single-user mode with minimal services (networking typically disabled)
- **Runlevels 2-5**: Various multi-user operational modes with full service availability
- **Runlevel 6**: System reboot/restart

The rc scripts are configured to execute specific services based on the selected runlevel, ensuring appropriate system functionality for each operational state.

Managing Startup Services with update-rc.d

You can customize which services automatically start during system boot by using the update-rc.d command. This utility allows you to add new services to or remove existing services from the rc.d startup sequence. The command syntax is straightforward and follows this pattern:

update-rc.d [service-name] [action]

Practical Example: Configuring PostgreSQL Auto-Start

Consider a scenario where you want PostgreSQL database service to launch automatically every time your system boots, ensuring it's immediately available for use with penetration testing frameworks like Metasploit for storing security assessment data.

First, verify whether PostgreSQL is currently running by using the process status command combined with a text filter:

```
ps aux | grep postgresql
```

If the output only shows the grep command itself, this indicates PostgreSQL is not currently active on the system.

To configure PostgreSQL for automatic startup, execute:

```
update-rc.d postgresql defaults
```

This command adds the necessary entry to the rc.d configuration file. The changes take effect after the next system reboot.

Following a system restart, running the same process check command will reveal that PostgreSQL is now running automatically without any manual intervention, ready for immediate use with your security testing tools.